Ejercicio 12: dónde encontrar cada cosa

Guía de consulta para explicar el login, la sesión y el ABM de renglones de factura. Código actualizado a JavaScript puro con AJAX (fetch), sin jQuery.

Buscá lo que te piden en la primera columna. La última columna indica el renglón de inicio; activá los números de línea en el editor. Esta edición corresponde al código comentado para la exposición.

Si te piden...	Función o evento	Archivo	Línea
Abrir Modificar y llenar campos	completarFormularioModificacion()	abm.js	313
Enviar la modificación	guardarModificacion()	abm.js	339
Abrir Alta registro	onclick de btnAlta	abm.js	62
Prefijo ART- y ejemplos del alta	value y placeholder	abm_template.php	93
Enviar el alta	guardarAlta()	abm.js	325
Borrar un registro	eliminarRenglon()	abm.js	353
Ver un PDF en vista previa	mostrarPdf()	abm.js	369
Ubicar el visor PDF en el HTML	etiqueta iframe	abm_template.php	152
Crear las filas y sus botones	dibujarTabla()	abm.js	270
Cargar datos y aplicar filtros	cargarDatos()	abm.js	238
Vaciar solo la tabla	onclick de btnVaciar	abm.js	24
Limpiar los filtros	onclick de btnLimpiar	abm.js	29
Ordenar por un encabezado	onclick de encabezados	abm.js	37
Esperar 300 ms al escribir	programarFiltro()	abm.js	104
Evitar factura + artículo repetido	validarDuplicadoEnFactura()	abm.js	159
Habilitar Enviar Alta	validarFormularioAlta()	abm.js	225
Habilitar Enviar Modificación	validarFormularioModificacion()	abm.js	232
Traer unidades de medida	cargarUnidades()	abm.js	184
Hacer la petición AJAX	enviarPeticionAjax()	abm.js	113
Cerrar sesión desde el ABM	onclick de btCierraSesion	abm.js	18
Comprobar usuario y contraseña	autenticacion()	ingresoAlSistema.php	28
Conectar con la base de datos	getConnection()	Database.php	21
Proteger el acceso al módulo	control de sesión	manejoSesion.inc	9

Ejemplo: "Mostrame el botón Modificar". Abrí **abm.js**: el clic se asigna en el renglón 296. Abre la ventana y llama a **completarFormularioModificacion()**, en el 313. Al guardar se ejecuta **guardarModificacion()**, en el 339.

Rutas: abm.js está en ejerciciosPHP/12-sesion/appModulo1/. La plantilla está en appModulo1/templates/. Database.php está dentro de appModulo1/models/. ingresoAlSistema.php y manejoSesion.inc están directamente en 12-sesion/. Los números corresponden al código local de esta edición; al agregar líneas pueden cambiar.

2. Cómo se reparte el trabajo

Un renglón de factura es un artículo dentro de una factura. La clave que lo identifica es el par **nroFactura + codArticulo**. El mismo artículo puede estar en otra factura. El importe se calcula con cantidad por precio unitario.

Archivo o carpeta dentro de 12-sesion/	Responsabilidad
formularioDeLogin.html	Formulario que envía usuario y contraseña por POST.
ingresoAlSistema.php	Autentica, incrementa el contador y guarda los datos de sesión.
manejoSesion.inc	Impide continuar si no existe el identificativo de sesión.
appModulo1/index.php	Incluye la protección y pide a la vista que muestre el tablero.
appModulo1/templates/abm_template.php	HTML: controles, tabla, formularios, ventanas y el iframe del PDF.
appModulo1/estilo.css	Presentación: colores, tamaños y fichas para pantallas chicas.
appModulo1/abm.js	Eventos, validación en el navegador, AJAX y actualización del HTML.
appModulo1/alta.php, modi.php, baja.php	Reciben los datos del navegador y llaman a la vista.
appModulo1/salidaJsonRenglones.php y salidaJsonUnidades.php	Devuelven datos JSON para la tabla y los selects.
appModulo1/traeBinario.php	Pide a la vista el PDF en Base64.
appModulo1/views/RenglonesView.php	Conecta los PHP de entrada con el modelo y prepara la respuesta.
appModulo1/models/RenglonesModel.php	Ejecuta las consultas SQL y procesa datos y archivos.
appModulo1/models/Database.php	Abre o reutiliza PDO y registra errores.

Palabras que aparecen en el código

DOM: los elementos HTML que JavaScript puede leer o modificar. **Evento:** una acción como clic, cambio de un campo o envío de formulario. **AJAX:** una petición al servidor que permite actualizar parte de la página sin recargarla.

JSON: texto con datos estructurados; JSON.parse lo convierte a un objeto. **FormData:** reúne los campos y archivos de un formulario. **Callback:** una función que se pasa como argumento para que otra la ejecute más tarde.

Modelo: código que consulta o modifica la base. **Vista:** en este ejercicio coordina llamadas y respuestas. **Plantilla:** el HTML que se muestra. El patrón del proyecto se llama MVT.

3. Arranque y botones de la pantalla

Cuando el HTML termina de cargarse

abm.js, renglones 8 - 101. DOMContentLoaded ejecuta la función anónima de arranque. Dentro se guardan las referencias al cuerpo de la tabla y al código del artículo, se prepara el orden inicial y se deshabilitan los envíos hasta que los formularios sean válidos.

También se piden las unidades de medida y la lista de claves existentes. Pedir esa lista no llena la tabla visible: sirve para validar el alta.

Evento en abm.js	Líneas	Qué ejecuta
Cierra Sesión	18-20	Navega a ../destruirsesion.php.
Cargar datos	21-23	cargarDatos(false): muestra las alertas didácticas y carga la tabla.
Vaciar datos	24-28	Escribe Sin datos y pone el contador visible en cero. No borra la base.
Limpiar filtros	29-35	Vacía los cinco campos de filtro y llama a cargarDatos(true).
Clic en un encabezado	36-41	Copia data-campo al input orden y recarga en silencio.
Escribir número, código o descripción	43-53	Programa el filtro; Enter cancela la espera y carga enseguida.
Cambiar unidad o fecha	54-60	Cancela el temporizador y aplica el filtro inmediatamente.
Alta registro	62-75	Resetea el formulario, limpia el error, actualiza claves, completa unidades y fecha, y abre el modal.
Cambiar campos del alta	76-81	Ejecuta validarFormularioAlta en input, change, keyup y blur.
Cambiar campos de modificación	82-86	Ejecuta validarFormularioModificacion en input, change y keyup.
Enviar formulario de alta	88-96	Evita la recarga; verifica duplicados y checkValidity. Solo si es válido llama a guardarAlta().
Enviar formulario de modificación	97-100	Evita la recarga y llama a guardarModificacion(). La validación nativa actúa al enviar el formulario.

Por qué hay funciones sin nombre

En una asignación como **boton.onclick = function() { ... }**, la función se llama anónima. El navegador la ejecuta al hacer clic. Para ubicarla buscá el ID del botón. Las funciones como guardarModificacion() sí tienen nombre y pueden llamarse desde otros lugares.

Las variables que se comparten

datosUnidades conserva las unidades. **clavesRenglonesExistentes** es un arreglo de claves. **temporizadorFiltro** guarda el temporizador. **ultimaPeticionListado** identifica la última búsqueda. **cuerpoTabla** y **campoCodigoArticuloAlta** apuntan a elementos del HTML. Están al principio de abm.js, renglones 1-6.

4. AJAX, búsqueda y tabla

enviarPeticionAjax()

appModulo1/abm.js | renglones **113-126**

Centraliza las peticiones con fetch, como en el ejercicio 10. Recibe la URL del PHP, los datos, una función para procesar la respuesta y una función opcional para los errores. async permite usar await dentro de esta función.

Renglón de abm.js	Explicación
114	Prepara una petición GET como opción inicial.
115	Si hay datos para enviar, cambia a POST y coloca esos datos en body. Si datos es null, conserva GET.
120	fetch inicia la petición. await espera su respuesta dentro de esta función, sin bloquear los otros eventos de la página.
121	respuesta.text() lee el cuerpo recibido como texto. También se espera con await.
122	Entrega el texto a la función que lo procesará. Los listados usan JSON.parse; las operaciones de escritura muestran la traza.
123	catch recibe los fallos de red o de procesamiento y los pasa a alError cuando existe.

URLSearchParams viaja como campos de formulario. FormData puede incluir el archivo PDF. fetch prepara el Content-Type correspondiente; para el archivo no se escribe ese encabezado a mano.

Ejemplo: cargarUnidades pasa una función que hace JSON.parse; guardarModificacion pasa otra que muestra el texto recibido. Esas funciones se ejecutan después de que responde el servidor. AJAX describe esta comunicación; fetch es la API que usa el código.

cargarDatos()

appModulo1/abm.js | renglones **238-267**

Lee el orden y los cinco filtros, arma el POST a salidaJsonRenglones.php y muestra Esperando respuesta mientras llega. Después convierte el JSON, llama a dibujarTabla y actualiza la cantidad del pie.

Con **silencioso = true** no aparecen alertas. Con false, se muestra el cuerpo enviado y la respuesta recibida. Si no hay filtros, se actualiza además el arreglo de claves completas para validar duplicados.

El contador ultimaPeticionListado aumenta con cada búsqueda. Si una respuesta vuelve con un número anterior, se descarta. Así una búsqueda lenta no pisa el resultado más reciente.

programarFiltro()

appModulo1/abm.js | renglones **104-109**

Cancela el temporizador anterior y programa cargarDatos(true) para dentro de 300 ms. Si seguís escribiendo, vuelve a empezar la espera. Evita pedir datos con cada tecla.

dibujarTabla()

appModulo1/abm.js | renglones **270-310**

Vacía el tbody y crea una fila por registro. Cada fila tiene ocho celdas de datos y tres acciones. Los valores se escriben con textContent para mostrarlos como texto, aunque contengan signos de HTML. Si no hay filas, muestra Sin resultados.

Las acciones se conectan en los renglones **290** (PDF), **296** (Modificar) y **306** (Borrar). Cada función conserva el renglón al que pertenece.

5. Validaciones y unidades de medida

guardarClavesExistentes()

appModulo1/abm.js | renglones **129-138**

Recibe las filas y reconstruye un arreglo de claves. Por ejemplo, factura 1001 y artículo ART-02 forman 1001||ART-02. trim elimina espacios al principio y al final; toUpperCase permite comparar sin distinguir mayúsculas.

actualizarClavesExistentes()

appModulo1/abm.js | renglones **141-156**

Pide todos los renglones por POST, sin filtros, y llama a guardarClavesExistentes. Si el alta está abierta, vuelve a validar sus campos. El parámetro alTerminar es opcional: alta y baja lo usan para recargar la tabla después de sincronizar. Si falla esta consulta auxiliar, la recarga continúa.

validarDuplicadoEnFactura()

appModulo1/abm.js | renglones **159-181**

Lee la factura y el artículo del alta. Si la clave ya está en el arreglo, setCustomValidity marca el código como inválido y muestra el mensaje. Si no está, limpia ese error. Cambiar la factura permite repetir el mismo artículo en otra factura.

Esta validación usa los datos cargados en el navegador. El modelo PHP vuelve a comprobar el par mediante existeRenglon antes de insertar. El control del navegador no reemplaza al del servidor.

validarFormularioAlta()

appModulo1/abm.js | renglones **225-229**

Primero valida el par factura + artículo. Después usa checkValidity() del formulario. Si devuelve true, disabled queda false y el botón Enviar Alta se habilita.

validarFormularioModificacion()

appModulo1/abm.js | renglones **232-235**

Aplica checkValidity() al formulario de modificación para habilitar o deshabilitar su botón de envío. La clave no se edita en ese formulario.

cargarUnidades()

appModulo1/abm.js | renglones **184-192**

Hace GET a salidaJsonUnidades.php, muestra el texto en una alerta, convierte el JSON y guarda las unidades para los selects.

agregarOpcionesUnidades()

appModulo1/abm.js | renglones **195-208**

Recibe un select, el arreglo de unidades y un valor opcional para dejar seleccionado. Crea una option por unidad y le asigna value, texto visible y clase.

llenarFiltroUnidades()

appModulo1/abm.js | renglones **211-215**

Conserva la primera opción del filtro, que representa todas las unidades, y agrega las demás.

llenarSelectorUnidades()

appModulo1/abm.js | renglones **218-222**

Vacía el select de un formulario, agrega la opción -- Seleccione UM -- y después las unidades. Al modificar recibe el código de unidad del registro para dejarlo seleccionado.

6. Modificar y dar de alta

Si te dicen: "Mostrame la función de Modificar"

1. Abrí **appModulo1/abm.js**, renglón **296**. Es el clic de Modificar dentro de dibujarTabla. Primero abre el modal; después llama a completarFormularioModificacion(renglon).

2. Mostrá **completarFormularioModificacion()**, renglones **313-322**. Es la función que pone los datos de la fila en los campos.

3. Si te piden guardar los cambios, mostrá **guardarModificacion()**, renglones **339-350**. Esa es la función que los manda al servidor.

completarFormularioModificacion()

appModulo1/abm.js | renglones **313-322**

Copia factura, fecha, artículo, descripción, cantidad y precio en el formulario de modificación. La fecha se recorta a sus primeros diez caracteres para usar AAAA-MM-DD. Vuelve a llenar el select de unidades y deja seleccionada la del registro. Al final valida el formulario.

El formulario está en **appModulo1/templates/abm_template.php**, renglón **114**. Factura y artículo tienen readonly: se mandan para localizar la fila, pero el usuario no puede cambiarlos en ese formulario.

guardarModificacion()

appModulo1/abm.js | renglones **339-350**

Crea un FormData con el formulario de modificación y lo envía por POST a modi.php. Al recibir la respuesta, muestra la traza en una alerta y en el modal, cierra el formulario y llama a cargarDatos(true). Si elegiste un archivo PDF, también viaja dentro del FormData.

Recorrido del servidor: **modi.php** llama a **procesarModi()** (appModulo1/views/RenglonesView.php, renglón 46), que delega en **actualizarRenglon()** (appModulo1/models/RenglonesModel.php, renglón 520). Allí está el UPDATE.

guardarAlta()

appModulo1/abm.js | renglones **325-336**

Crea un FormData con el formulario de alta y lo envía a alta.php. Muestra la respuesta, cierra el alta, vuelve a pedir la lista de claves y luego recarga la tabla. Esa sincronización permite detectar el registro nuevo en las siguientes altas.

Ejemplos y prefijo del alta: están en **appModulo1/templates/abm_template.php**, desde el renglón **89**. El atributo placeholder muestra un ejemplo cuando el campo está vacío. En código de artículo, value="ART-" deja ese prefijo escrito. Al abrir el alta, form.reset() restaura ese valor; el usuario completa el resto del código.

Recorrido del servidor: **alta.php** llama a **procesarAlta()** (appModulo1/views/RenglonesView.php, renglón 38). La vista llama a **insertarRenglon()** (appModulo1/models/RenglonesModel.php, renglón 400), que comprueba el duplicado y ejecuta el INSERT.

Lo que podés explicar en voz alta

"El clic de Modificar abre y completa el formulario. Cuando envío ese formulario, guardarModificacion toma sus campos con FormData y hace una petición AJAX a modi.php. La vista llama al modelo, que actualiza la fila usando número de factura y código de artículo. La respuesta vuelve al modal y se recarga la tabla".

7. Borrar, vista previa PDF y ventanas

eliminarRenglon()

appModulo1/abm.js | renglones **353-366**

Recibe factura y artículo. confirm pregunta si querés borrar ese renglón; cancelar termina la función sin enviar nada. Si aceptás, arma FormData con esos dos campos y lo manda por POST a baja.php. Muestra la respuesta y actualiza las claves y la tabla.

En el servidor, **procesarBaja()** (appModulo1/views/RenglonesView.php, renglón 54) llama a **eliminarRenglon()** (appModulo1/models/RenglonesModel.php, renglón 621). El DELETE usa los dos componentes de la clave.

mostrarPdf()

appModulo1/abm.js | renglones **369-384**

Hace GET a traeBinario.php con nroFactura y codArticulo. encodeURIComponent protege esos valores dentro de la URL. La respuesta trae documentoPdf en Base64; si existe, se carga en el iframe y se abre la ventana. Si falta, muestra una alerta.

La línea que muestra el archivo: abm.js, renglón **376**. Asigna al src el prefijo data:application/pdf;base64, seguido del contenido recibido.

El espacio donde se ve: appModulo1/templates/abm_template.php, renglón **152**. Ese iframe usa el visor de PDF del navegador; abm.js abre la ventana y le indica qué documento cargar.

De dónde viene: traeBinario.php llama a emitirBinarioPdf() de la vista, y esta llama a getPdfBase64() del modelo. Ese método consulta el binario de la fila y aplica base64_encode. La explicación de los métodos PHP está en las páginas siguientes.

abrirVentana()

appModulo1/abm.js | renglones **387-392**

Agrega contenedorPasivo al tablero para bloquear los clics de atrás, busca la ventana por ID, le asigna visibility visible y agrega la clase activo.

cerrarVentana()

appModulo1/abm.js | renglones **395-400**

Oculta la ventana, quita activo y retira contenedorPasivo del tablero. Las cuatro funciones de cierre específicas llaman a esta función común.

mostrarRespuestaServidor()

appModulo1/abm.js | renglones **403-408**

Abre el modal de respuesta y coloca el texto devuelto por el PHP. Se usa después de alta, modificación y baja.

Cierre específico en abm.js	Línea	Ventana que cierra
cerrarFormularioAlta()	411	Formulario de alta.
cerrarFormularioModificacion()	413	Formulario de modificación.
cerrarRespuestaServidor()	415	Respuesta del servidor.
cerrarVistaPreviaPdf()	417	Visor PDF; además vacía el src del iframe.

8. Login, sesión y conexión a la base

autenticacion()

ingresoAlSistema.php | renglones **28-70**

Recibe usuario, contraseña y una variable de contador por referencia. Crea PDO, calcula el hash SHA-256 de la contraseña y busca el usuario con una consulta preparada. Si el hash coincide, incrementa el contador, lo devuelve por referencia y retorna true. Si no coincide o hay un error de base, retorna false. El código que llama a esta función guarda después las variables de sesión.

En **ingresoAlSistema.php**, renglón **10**, se guarda el identificativo. Debajo se guardan login y ContadorDeSesion. session_start retoma o inicia la sesión PHP; session_create_id genera la cadena usada como identificativo en este ejercicio.

infoDeSesion()

ingresoAlSistema.php | renglones **103-115**

Lee el identificativo, el usuario, el contador y la IP, y los imprime en un recuadro HTML. Usa htmlspecialchars para mostrar los datos como texto. No vuelve a consultar la base.

manejoSesion.inc: la protección de los PHP

Es un script incluido, no una función. En los renglones 9-11 abre o retoma la sesión si hace falta. En los renglones 18-20 verifica identificativo. Si no existe o está vacío, redirige a ../index.php y ejecuta exit. Así no continúa el PHP protegido.

Cerrar sesión

El botón del ABM navega a **12-sesion/destruirsesion.php**. Ese script vacía $_SESSION, hace caducar la cookie si corresponde, ejecuta session_destroy y redirige al login. También existe una copia en appModulo1 usada por otros botones; sus rutas relativas al formulario son distintas.

getConnection()

appModulo1/models/Database.php | renglones **21-61**

Es un método estático de Database. La primera llamada busca los datos de conexión, crea un PDO y lo guarda en self::$pdo. Las siguientes llamadas dentro de la misma petición reutilizan ese objeto. Configura excepciones, resultados asociativos y consultas preparadas nativas. Si falla, registra el error y devuelve null.

logError()

appModulo1/models/Database.php | renglones **10-19**

Agrega una línea con fecha y mensaje a los logs del módulo y de 12-sesion. FILE_APPEND escribe al final y LOCK_EX bloquea la escritura mientras agrega la línea.

Los archivos datosConexionBase.php contienen la configuración de conexión. La guía no copia credenciales. Los renglones de Database.php e ingresoAlSistema.php corresponden a las copias locales que estaban al generar esta edición.

9. Modelo: columnas, unidades y consultas

Archivo de esta página: **appModulo1/models/RenglonesModel.php**. Los métodos privados ayudan a la propia clase; los públicos son los que llama la vista. Las consultas se ejecutan en el servidor.

__construct()

appModulo1/models/RenglonesModel.php | renglones **11-13**

Pide la conexión a Database::getConnection y la guarda en $this->pdo.

getPdo()

appModulo1/models/RenglonesModel.php | renglones **16-18**

Devuelve la conexión guardada en el modelo.

buscarColumna()

appModulo1/models/RenglonesModel.php | renglones **25-68**

Busca el nombre real de un campo entre los nombres de columnas existentes. Prueba coincidencia exacta, por palabra y por subcadena. Anota las columnas elegidas en $usadas para no asignar una misma columna a dos datos diferentes.

detectarColumnas()

appModulo1/models/RenglonesModel.php | renglones **74-176**

Lee SHOW COLUMNS de renglones_factura y arma el mapa de nombres reales para factura, fecha, artículo, descripción, unidad, cantidad, precio, importe y PDF. También detecta si la unidad es numérica y si el importe es generado por MySQL. Guarda el resultado en cacheColumnas para reutilizarlo durante la petición.

obtenerInfoUnidades()

appModulo1/models/RenglonesModel.php | renglones **182-203**

Lee las unidades y arma equivalencias entre ID, código y descripción. Permite mostrar un código como UN aunque la base guarde un ID numérico.

resolverValorUmParaBD()

appModulo1/models/RenglonesModel.php | renglones **209-234**

Si la columna de unidad espera un entero, busca el ID que corresponde al código o descripción recibido. Si espera texto, conserva el valor de texto.

getUnidades()

appModulo1/models/RenglonesModel.php | renglones **264-273**

Consulta SELECT * FROM unidades_medida y devuelve las filas. Si no hay conexión o falla la consulta, devuelve un arreglo vacío.

getRenglones()

appModulo1/models/RenglonesModel.php | renglones **276-397**

Valida el campo de orden contra una lista permitida. Lee los cinco filtros y arma condiciones SQL con parámetros. Busca coincidencias parciales para número, código y descripción; resuelve equivalencias para unidad y filtra fecha por su prefijo. Ejecuta el SELECT, normaliza las unidades y devuelve renglones y cantidad.

Las condiciones se unen con AND: si completás varios filtros, una fila debe cumplirlos todos. El ORDER BY usa una columna del mapa permitido y orden ascendente. Los valores de búsqueda se pasan mediante parámetros de PDO.

10. Modelo: altas, cambios, bajas y PDF

existeRenglon()

appModulo1/models/RenglonesModel.php | renglones **240-261**

Consulta si ya existe la combinación factura + artículo. Limpia espacios y compara sin distinguir mayúsculas. Devuelve un booleano. El alta llama a este método antes del INSERT.

insertarRenglon()

appModulo1/models/RenglonesModel.php | renglones **400-517**

Recibe el arreglo de datos, una traza por referencia y el PDF opcional. Limpia los campos, convierte cantidad y precio a números y calcula el importe. Rechaza la clave repetida. Después detecta las columnas, resuelve la unidad y ejecuta un INSERT preparado.

No incluye el importe si la base lo genera automáticamente. Si llegó un archivo y existe la columna PDF, lee el archivo temporal y guarda el binario con PDO::PARAM_LOB. El método agrega mensajes a $traza para que vuelvan al navegador.

actualizarRenglon()

appModulo1/models/RenglonesModel.php | renglones **520-618**

Prepara el UPDATE para fecha, descripción, unidad, cantidad, precio e importe cuando corresponda. Factura y artículo se usan en el WHERE para localizar el renglón; no forman parte de los campos editables. Si llegó otro PDF, actualiza su columna.

eliminarRenglon()

appModulo1/models/RenglonesModel.php | renglones **621-648**

Ejecuta DELETE con dos condiciones: número de factura y código de artículo. Prepara la consulta, vincula ambos parámetros y ejecuta. Escribe el resultado o el error en la traza.

getPdfBase64()

appModulo1/models/RenglonesModel.php | renglones **651-671**

Busca el PDF usando la misma clave compuesta. Si encuentra contenido, devuelve base64_encode del binario. Si falta conexión, columna PDF, documento o hay error, devuelve null.

Qué significa la traza por referencia

En una firma como **insertarRenglon($datos, &$traza, $archivoPdf)**, el & permite modificar la variable de quien llamó al método. La vista crea $traza vacía, el modelo le agrega mensajes y la vista hace echo para enviarla al navegador.

Qué parte guarda cada cosa

FormData transporta campos y archivo desde el navegador. **$_POST** contiene los campos en PHP. **$_FILES** contiene la información del archivo temporal. **PDO** ejecuta el SQL. El PDF se guarda como binario y se transforma a Base64 cuando se pide para la vista previa.

La confirmación de Borrar se hace en JavaScript. El borrado real ocurre recién al ejecutar eliminarRenglon en el servidor. Vaciar datos solo cambia lo que se muestra en pantalla.

11. Vista y archivos PHP de entrada

La clase de coordinación está en **appModulo1/views/RenglonesView.php**. Su constructor crea RenglonesModel. render incluye la plantilla HTML.

Método de RenglonesView	Líneas	Responsabilidad
__construct()	9-11	Crea el modelo.
render()	14-16	Incluye templates/abm_template.php.
emitirJsonUnidades()	19-26	Pide unidades y devuelve {unidades: [...]}. Tiene sleep(2) didáctico.
emitirJsonRenglones()	29-35	Pasa filtros y orden al modelo; devuelve renglones y cantidad. Tiene sleep(1).
procesarAlta()	38-43	Crea traza, llama a insertarRenglon e imprime la traza.
procesarModi()	46-51	Crea traza, llama a actualizarRenglon e imprime la traza.
procesarBaja()	54-59	Crea traza, llama a eliminarRenglon e imprime la traza.
emitirBinarioPdf()	62-70	Llama a getPdfBase64 y devuelve JSON con documentoPdf.

Cada petición entra por uno de estos archivos

PHP en appModulo1/	Lee y llama a...	Línea de la llamada
index.php	Incluye manejoSesion.inc; crea la vista y llama a render().	17
alta.php	Arma $datos desde $_POST y toma $_FILES. Llama a procesarAlta().	27
modi.php	Arma los mismos campos y el archivo opcional. Llama a procesarModi().	27
baja.php	Lee factura y artículo del POST. Llama a procesarBaja().	18
salidaJsonRenglones.php	Lee orden y cinco filtros del POST. Llama a emitirJsonRenglones().	24
salidaJsonUnidades.php	Llama a emitirJsonUnidades().	15
traeBinario.php	Acepta factura y artículo por GET o POST. Llama a emitirBinarioPdf().	18

Estos PHP incluyen manejoSesion.inc antes de continuar. Los listados y el PDF responden JSON; alta, modificación y baja devuelven la traza como texto. Por eso enviarPeticionAjax lee el cuerpo con respuesta.text() y cada función decide si usa JSON.parse o muestra el texto.

Otros scripts del ejercicio

seedUsuario.php: prepara usuarios de prueba en la base. **descargarErrores.php:** reúne líneas de logs sin repetirlas y descarga errores.txt. El enlace del tablero usa la copia ubicada en 12-sesion. No son funciones de abm.js.

12. Índice completo de abm.js

Este índice sigue el orden del archivo. Los rangos incluyen la declaración y su cierre. El bloque de arranque está en los renglones 8-101; las funciones con nombre están debajo.

Función en appModulo1/abm.js	Renglones	Para qué sirve
programarFiltro()	104-109	Espera al terminar de escribir.
enviarPeticionAjax()	113-126	Envía la petición con fetch.
guardarClavesExistentes()	129-138	Reconstruye el arreglo de claves.
actualizarClavesExistentes()	141-156	Actualiza las claves desde el servidor.
validarDuplicadoEnFactura()	159-181	Marca el duplicado factura + artículo.
cargarUnidades()	184-192	Pide y conserva las unidades.
agregarOpcionesUnidades()	195-208	Agrega opciones a un select.
llenarFiltroUnidades()	211-215	Llena el filtro de unidades.
llenarSelectorUnidades()	218-222	Llena el select de alta o modificación.
validarFormularioAlta()	225-229	Controla el botón Enviar Alta.
validarFormularioModificacion()	232-235	Controla el botón Enviar Modificación.
cargarDatos()	238-267	Pide el listado filtrado y ordenado.
dibujarTabla()	270-310	Crea filas, celdas y acciones.
completarFormularioModificacion()	313-322	Copia una fila al formulario de edición.
guardarAlta()	325-336	Envía el formulario nuevo.
guardarModificacion()	339-350	Envía los cambios.
eliminarRenglon()	353-366	Confirma y solicita el borrado.
mostrarPdf()	369-384	Pide y muestra la vista previa.
abrirVentana()	387-392	Muestra una ventana.
cerrarVentana()	395-400	Oculta una ventana.
mostrarRespuestaServidor()	403-408	Presenta la traza del PHP.
cerrarFormularioAlta()	411	Cierra el formulario de alta.
cerrarFormularioModificacion()	413	Cierra el formulario de edición.
cerrarRespuestaServidor()	415	Cierra el mensaje del servidor.
cerrarVistaPreviaPdf()	417-420	Cierra el visor y limpia el iframe.

Para ubicar una función, buscá su nombre con la búsqueda del editor. Para un evento anónimo, buscá el ID del botón. Los botones PDF, Modificar y Borrar se crean en dibujarTabla; los botones generales están en DOMContentLoaded.